Actor-Critic síncrono (A2C) — relatório experimental

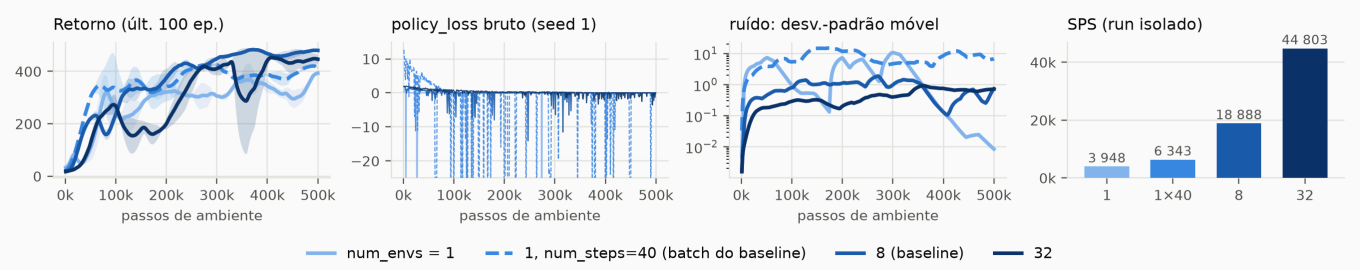
Vinícius Tormin e **Bruno Calura** · Aprendizado por Reforço, UFG · CartPole-v1, 500k passos, 2 seeds por configuração
Fork com `algorithms/a2c.py` e `networks/discrete_actor_critic.py` completos: github.com/viniciustormin/t-zero-learning

Protocolo. Baseline = `configs/a2c_cartpole.yml` sem alterações (`num_envs=8`, `num_steps=5`, `ent_coef=0,01`, `use_baseline=true`); cada varredura muda um hiperparâmetro via `--override`, **seeds 1 e 2 em todas as configurações**. As previsões foram escritas e commitadas antes de qualquer run completo (`report/predictions_a2c.md`, commit `bcad857`, anterior ao da geração das figuras). Linha grossa = média entre seeds, faixa = envoltória dos dois; cor mais escura = valor maior do hiperparâmetro. Curvas suavizadas por EMA, **exceto** `policy_loss`, desenhado bruto — o ruído dele é o dado. Sem conta no W&B (modo anônimo descontinuado, erro 401): `WANDB_MODE=offline`, e as figuras vêm do espelho local de cada `wandb.log` (`tools/train_csv.py` → `sweep_results_a2c/<config>__seed<N>/metrics.jsonl`); os diretórios offline ficam em `wandb/` e sincronizam com `wandb sync`. **Atenção ao ler `eval`:** a avaliação final de 10 episódios do framework **amostra da política** (`evaluate(..., deterministic=False)`), não é greedy — uma política mais estocástica é penalizada nela.

Q1 — Número de atores em paralelo (`num_envs`)

Previsão. Com 1 ambiente o lote são 5 transições consecutivas do mesmo episódio: gradiente de variância enorme e correlacionado, `policy_loss` muito mais ruidoso e retorno instável. Com 32 ambientes o gradiente fica próximo do verdadeiro, mas como `num_iterations = 500k/(num_envs·num_steps)` são só 3125 updates contra 12500 do baseline, esperávamos aprendizado **mais lento por passo de ambiente**. Incluímos o controle `num_envs=1, num_steps=40`, com **o mesmo `batch_size` (40) e o mesmo número de updates** do baseline: o que sobrar é descorrelação entre atores, não tamanho de lote.

Varredura. `num_envs` ∈ {1, **8 (baseline)**, 32} + controle `1×40`, seeds 1 e 2. O `SPS` foi re-medido com **um run por vez** (`tools/bench_sps_a2c.py`): o sweep roda 6 jobs simultâneos e o `charts/SPS` de lá mede disputa de CPU, não throughput da configuração.



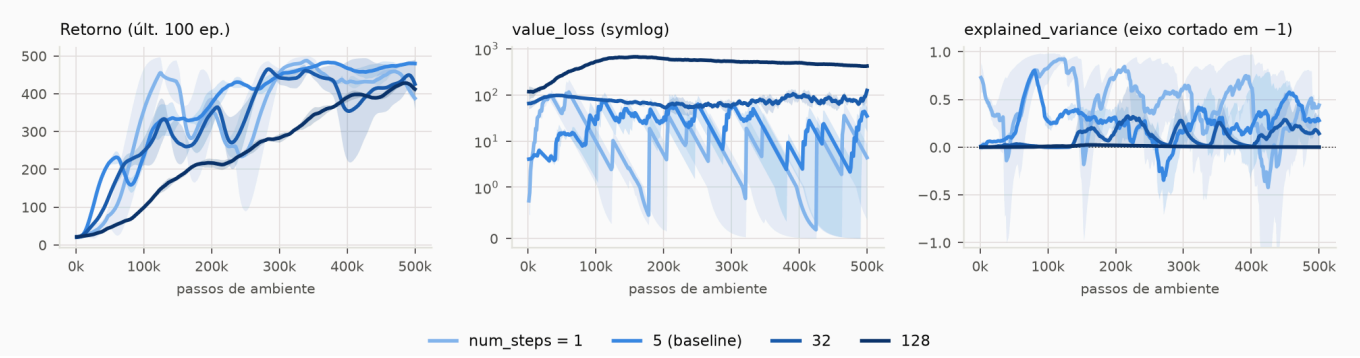
Explicação. O argumento do A3C é sobre **descorrelação**, e o controle `1×40` o isola: mesmo lote, mesmo número de updates, e ainda assim o desvio-padrão do `policy_loss` ao longo do run é **11,2 / 9,6** contra **1,4 / 1,3** do baseline — quase 8× mais ruído — com retorno final 412/423 contra 475/486. A única diferença é a origem das 40 transições: 40 passos consecutivos de um episódio contra 5 passos de 8 episódios independentes. Um lote correlacionado tem **tamanho efetivo de amostra** muito menor que 40, então cada update persegue o trecho de trajetória mais recente; o terceiro painel mostra o controle uma ordem de grandeza acima das demais o run inteiro.

O efeito de **tamanho de lote** é outro e aparece separado. Com `num_envs=32` o ruído é o menor de todos (0,7 / 0,8) e a curva a mais suave, mas o retorno em 500k passos fica **abaixo** do baseline (432/465): 3125 updates não bastam. E `num_envs=1` (lote de 5) é o caso degenerado — cinco estados quase idênticos do mesmo episódio têm retornos quase idênticos, então `advantage_std` colapsa (percentil 95 de **0,06**, contra **15,8** do baseline) e a `explained_variance` do lote explode para **–6,7 / –3,0** (o denominador é ≈0). O gradiente quase some: é por isso que o `policy_loss` bruto dele cola no zero no segundo painel — não é estabilidade, é ausência de sinal. Em throughput o paralelismo compra muito: **3 948 → 18 888 → 44 803 SPS** de 1 → 8 → 32 ambientes, 11,3× entre os extremos, porque o custo fixo de cada forward/backward é amortizado sobre mais transições. **Previsão: confirmada, com uma correção** — erramos ao supor que `num_envs=1` teria o `policy_loss` mais ruidoso em valor absoluto: ele tem os **pesos** degenerados, problema pior e de outra natureza.

Q2 — Horizonte do retorno de n passos (`a2c.num_steps`)

Previsão. `num_steps=1` dá o alvo $r + \gamma V(s')$: variância mínima, viés máximo; `num_steps=128` é quase Monte Carlo: viés quase nulo, variância alta. Esperávamos `value_loss` crescendo com `num_steps` só pela escala do alvo, `explained_variance` **enganosamente alta** em `num_steps=1`, e o melhor retorno no meio.

Varredura. `a2c.num_steps` ∈ {1, **5 (baseline)**, 32, 128}, seeds 1 e 2. O lote acompanha — 8, 40, 256 e 1024 transições, ou seja 62 500, 12 500, 1 953 e 488 updates em 500k passos.



Explicação. A `value_loss` atravessa três ordens de grandeza (mediana **0,0 → 0,0 → 77 → 505**), mas isso é quase todo escala: o alvo de 128 passos vale dezenas de unidades e o de 1 passo vale $r + \gamma V \approx 1 + V$. O painel que separa viés de variância é o da direita. Com `num_steps` grande a `explained_variance` fica **colada em 0,00** nos dois seeds — o crítico não explica nada da variância do retorno, porque o alvo Monte Carlo carrega o ruído das 128 escolhas estocásticas seguintes e a rede só consegue prever a média. O ator herda isso: a vantagem $R - V$ vira quase puro ruído, e o `advantage_mean` com `num_steps=128` fica em **+21,6 / +23,1** em vez de ≈0, sinal de que o crítico subestima sistematicamente um alvo que não acompanha.

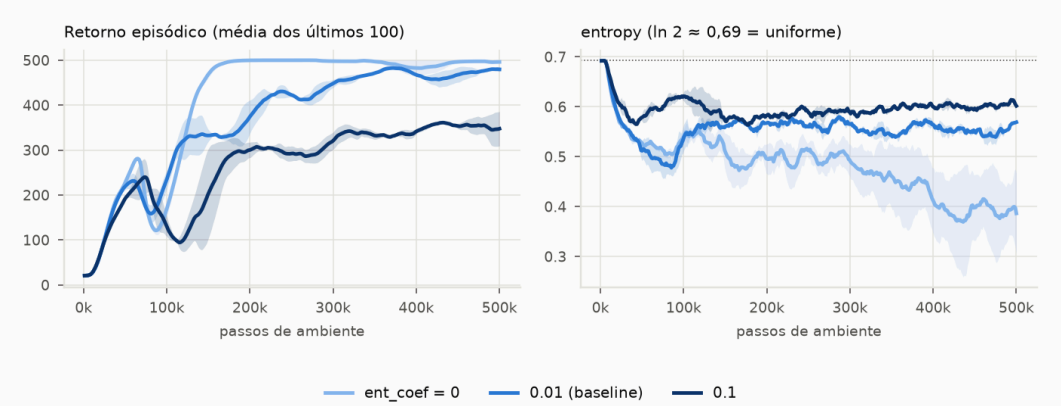
E aqui está a armadilha. Com `num_steps=1` a `explained_variance` é a **mais alta de toda a varredura** (mediana 0,69 / 0,96) e a `value_loss` a menor — e mesmo assim esse é o pior retorno de treino do grupo (**372 / 402** contra 475/486). O motivo é que o alvo **é** $r + \gamma V(s')$: o

crítico é pontuado contra a própria predição deslocada de um passo, então acerta por construção, não por saber o valor verdadeiro. Um crítico pode ter `explained_variance` ≈ 1 e estar completamente errado sobre V^{π}, porque o erro de bootstrap se propaga devagar demais para aparecer dentro do lote. **Previsão: confirmada.** O que não antecipamos foi a não-monotonicidade em `num_steps=32`, cujos seeds discordam fortemente (eval 191 e 486): com 1953 updates, a sorte da inicialização ainda pesa muito.

Q3 — Coeficiente de entropia (`a2c.ent_coef`)

Previsão. O bônus de entropia contrapõe o laço de realimentação positiva do gradiente de política ("ação amostrada com vantagem positiva → sua probabilidade sobe → é amostrada mais"). Com `ent_coef=0,1` esperávamos a entropia presa perto de $\ln 2 \approx 0,69$ e o retorno estacionando; com `ent_coef=0`, o CartPole funcionando mesmo assim, com entropia caindo mais cedo.

Varredura. `a2c.ent_coef` ∈ {0, **0,01 (baseline)**, 0,1}, seeds 1 e 2 em todas.

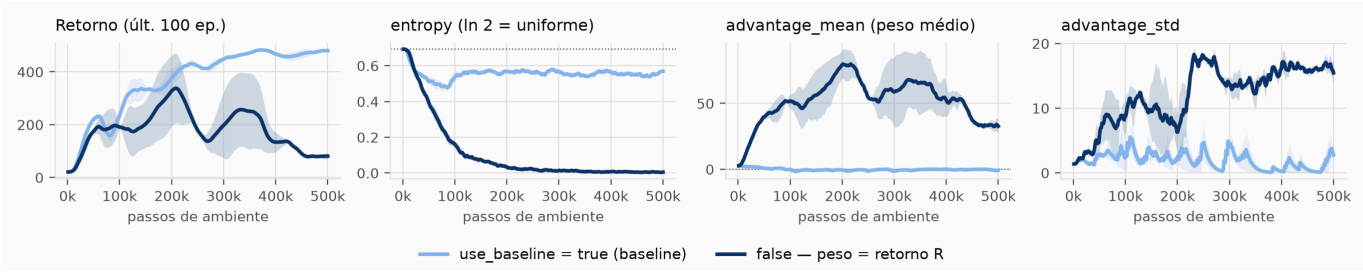


Explicação. Com `ent_coef=0,1` a entropia estaciona em **0,605 / 0,609**: numa política binária isso é uma divisão de ≈80/20, ou seja um quinto das ações continua sendo o lado errado o run inteiro, e o retorno estaciona em **386 / 305** por consequência direta. O termo $-0,1 \cdot H$ age como uma mola puxando os logits de volta ao uniforme, e no CartPole a política ótima é quase determinística — a mola compete com o objetivo em vez de ajudá-lo. Com `ent_coef=0` o treino não só funciona como dá o **melhor** resultado de toda a varredura (**498 / 494**, eval 500/500), com entropia caindo a 0,456 / 0,330 (≈90/10). O CartPole é benévolo exatamente nas três dimensões que barateiam a exploração: duas ações apenas (colapsar para a errada é punido de imediato), recompensa densa a cada passo (crédito sem atraso) e um ambiente que **gera estados novos sozinho** quando a política é ruim — o carrinho sai do centro e o agente vê estados que nunca visitaria de propósito. Com mais ações ou recompensa esparsa nada disso vale: o colapso prematuro fixaria a política numa sub-região antes de qualquer recompensa ser encontrada, e sem o bônus não haveria força para desfazê-lo. **Previsão: confirmada.** Note por que `ent_coef=0` também ganha na coluna `eval`: a avaliação amostra da política, então entropia menor significa menos ações erradas **também na avaliação** — parte dessa vantagem é artefato do protocolo, não da política aprendida.

Q4 — Ablação do baseline (`a2c.use_baseline=false`)

Previsão. Subtrair $V(s)$ não muda o gradiente esperado, mas muda a variância: esperávamos `advantage_mean` ≈ 0 com baseline e fortemente positivo sem ele, `advantage_std` na mesma ordem nos dois, e colapso **mais rápido** da entropia sem baseline — no CartPole toda recompensa é positiva, e um peso positivo sobe $\log \pi(a|s)$ de qualquer ação amostrada.

Varredura. `a2c.use_baseline` ∈ {**true (baseline)**, false}, seeds 1 e 2.



Explicação. Os painéis de baixo confirmam e quantificam o resultado teórico. Com baseline o peso do gradiente tem média **–0,4 / –1,7** e desvio **1,9 / 5,5**; sem baseline a média sobe para **+27,7 / +38,3** e o desvio para **14,1 / 17,8**. Como o gradiente esperado é o mesmo nos dois casos, o que mudou foi só o segundo momento do peso: $E[w^2] = \mu^2 + \sigma^2$ passa de ≈18 para ≈1375 — cerca de 75× mais variância no estimador para a mesma informação. O `advantage_mean` sem baseline é essencialmente o retorno descontado médio da política, que é exatamente o que $V(s)$ existe para cancelar.

A entropia mostra por que isso é fatal, e não apenas ineficiente. No CartPole a recompensa é +1 por passo, logo $R > 0$ **sempre**: sem baseline o peso é positivo em toda transição e o update sobe $\log \pi(a|s)$ de **qualquer** ação amostrada. A política deixa de aprender "o que foi melhor que a média" e passa a aprender "o que eu fiz" — um laço de realimentação puro, interrompido só quando não há mais o que reforçar. É o que se vê: a entropia despenca de $\ln 2$ para **0,002 / 0,005** em menos de 200k passos, contra 0,562 / 0,566 do baseline, e o retorno acompanha, caindo de um pico de ≈340 para **69 / 92**. É o mecanismo da Q3 sem nada que o contrabalance — e note que o `ent_coef=0,01` continua ligado aqui: o bônus não é forte o bastante contra um peso sempre positivo. O detalhe mais instrutivo está fora dos painéis: a `explained_variance` da ablação é **0,99**, a melhor de todo o trabalho. Com a política determinística e episódios de ≈70 passos o retorno é trivialmente previsível e o crítico o acerta — um crítico impecável e inútil, porque `use_baseline=false` faz com que ele não seja usado. **Previsão: confirmada no mecanismo, errada num detalhe.** Previamos `advantage_std` "na mesma ordem nos dois casos", com a piora vindo só do deslocamento da média; na prática o desvio também cresceu 3–8×, porque sem baseline o peso herda a variância **entre estados** do retorno R, que o baseline cancela junto com a média. O efeito real é portanto pior do que previamos, não melhor.

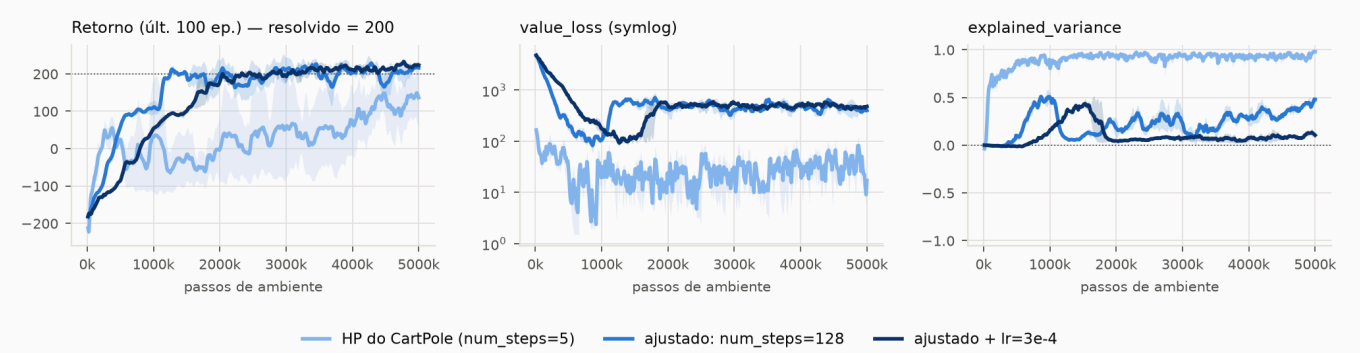
Avaliação final (10 episódios amostrando da política, seed 1 / seed 2), retorno de treino ao fim do run e diagnósticos

	base	envs=1	1×40	envs=32	steps=1	steps=32	steps=128	ent=0	sem base.
eval	**500/500**	412/180	459/248	500/425	500/500	191/486	316/394	500/500	65/93
retorno treino	**475/486**	400/398	412/423	432/465	372/402	380/491	406/429	498/494	69/92
entropy	**0,56**	0,56	0,59	0,56	0,39	0,52	0,51	0,39	0,003
expl.var med.	**0,46/0,25**	−6,7/−3,0	−0,1/0,0	0,24/0,04	0,69/0,96	0,00/0,03	0,00/0,00	0,40/0,68	0,99/0,99
desv.-pad. policy_loss	**1,4/1,3**	4,5/7,8	11,2/9,6	0,7/0,8	1,1/2,5	2,7/2,4	2,7/3,0	1,3/1,3	5,1/6,0

Extra — LunarLander-v3

O que mudou, e por quê. Partimos de `a2c_cartpole` e rodamos um piloto de 10 arms (2M passos, 1 seed, `tools/run_sweep_a2c.py --suite lunar`) antes de fixar a config. Nossa hipótese inicial era de **escala de recompensa**: retornos de −400 a +250 tornam a `value_loss` ordens de grandeza maior que a do CartPole, então `vf_coef·value_loss` dominaria o gradiente e `max_grad_norm=0,5` aniquilaria a parte do ator; baixamos `vf_coef` para 0,25, subimos o clipping para 1,0 e alargamos a rede para 128. **O piloto derrubou a hipótese:** as arms `ll_vf0.5` e `ll_clip0.5`, que desfazem exatamente essas duas mudanças, ficam dentro do ruído da config ajustada (eval 201 e 146 contra 185). Quem decide é o **horizonte n-step**: `num_steps` ∈ {5, 32, 128, 256} dá eval **114, −115, 185 e 209**. É a Q2 lida num ambiente onde ela morde — a recompensa que define o episódio (+100 por pousar, −100 por bater) chega **só no fim**, depois de centenas de passos de shaping. Com `num_steps=5` o alvo é quase todo $\gamma^5 V(s_{t+5})$, e o crédito pelo pouso só alcança o início do episódio depois de se propagar por dezenas de updates através de um crítico que ainda não sabe prevê-lo; com 128 passos, boa parte do episódio vê a recompensa terminal diretamente no alvo.

Resultado. Config final em `configs/a2c_lunarlander.yml`: `num_envs=16`, `num_steps=128`, `hidden=128`, `vf_coef=0,25`, `max_grad_norm=1,0`, `lr=7e-4`, 5M passos — contra os hiperparâmetros do CartPole intocados e contra `lr=3e-4`, 2 seeds cada.



Com `num_steps=128` o ambiente é **resolvido**: retorno de treino de **213 / 221** ao fim dos 5M passos (picos de 259 / 246), média **217** entre seeds, acima do limiar de 200; `lr=3e-4` chega ao mesmo lugar com menos dispersão (**217 / 230**). O controle com os hiperparâmetros do CartPole é o resultado mais instrutivo: ele **não** é incapaz — o seed 1 chega a **216** — mas o seed 2 estaciona em **56**, e é o painel da direita que diz por quê. A `explained_variance` do controle é **0,98 / 0,99**, contra **0,20 / 0,23** da config ajustada, e a `value_loss` dele é duas ordens de grandeza menor. É exatamente a armadilha da Q2, agora num ambiente que a pune: com um alvo de 5 passos o crítico é pontuado contra a própria predição e parece impecável, enquanto o crédito pelo pouso leva dezenas de updates para atravessar o episódio — metade dos seeds não chega lá.

O que ainda limita a config ajustada aparece no mesmo painel, com o sinal invertido: `explained_variance` de apenas **0,20** (e **0,05** com `lr=3e-4`) significa que, com 128 passos, o alvo é quase Monte Carlo e o crítico quase não reduz variância nenhuma — o que sustenta o aprendizado é o **baixo viés** do alvo, não a qualidade da baseline. Estamos pagando variância para comprar viés, e não temos como não pagar, porque `num_steps` é um único botão para os dois. O próximo passo natural seria GAE(λ), que interpola continuamente entre os dois regimes em vez de obrigar a escolher um deles.

Reprodução: `python tools/run_sweep_a2c.py` (20 runs, 5,5 min em CPU num M4 Pro, 6 simultâneos), `python tools/bench_sps_a2c.py`, `python tools/run_sweep_a2c.py --suite lunar`, `python tools/make_figures_a2c.py` e `python tools/build_report_a2c.py`. Implementação: `compute_n_step_returns` (varredura para trás com acumulador $(N,)$, `done` zerando o bootstrap da própria coluna), `compute_policy_loss` ($-\overline{\log \pi \cdot A}$ com o peso **destacado do grafo** — sem isso a perda de política também moveria $V(s)$, já que `values` é reusado pela perda do crítico no mesmo `backward`) e `DiscreteActorCritic.get_action_and_value` (`Categorical(logits=...)`, `sample()` ou argmax, ação dada devolvida inalterada). `python -m pytest tests/test_a2c.py`: 14/14.